The single-argument `profit` function was a good description of the profit for a given price of a single camera type. Adding more arguments made it more generally useful but obscured the meaning. How can you get a simple, single-argument `profit` function for different types of cameras?

```
photoProProfit 162
# 7672

pixelKingProfit 200
# 14860
```

How can you make this interface possible?

To make this interface possible, change the `profit` function to return another function that accepts the single argument:

```
profit = (overhead, costPrice, numberSold) ->
  revenue = (salePrice) ->
     (numberSold salePrice) * salePrice
  cost = (salePrice) ->
    overhead + (numberSold salePrice) * costPrice
  (salePrice) ->
    (revenue salePrice) - (cost salePrice)
```

Take overhead, costPrice, and numberSold as arguments.

Return a function that takes salePrice as an argument.

When invoked, this new `profit` function *returns* a function that will evaluate to the profit for a specific camera (this time it's a camera called the x1) when *it* is invoked:

```
x1Overhead = 140
x1CostPrice = 100
x1NumberSold = (salePrice) -> 50 + 20/10 * (200 - salePrice)

x1Profit = profit x1Overhead, x1CostPrice, x1NumberSold
x1Profit 162
# 7672
```

This photoProProfit function can be invoked whenever you need it.

Why is this technique called partial application? Because only *some* of the arguments to the function have been provided, so the arguments have been partially applied. In this case the `salePrice` argument hasn't yet been provided and hasn't yet been *applied*.

So far you've been working on defining `profit` without looking at the larger program that it's part of. What program is this `profit` function used for? Agtron is creating a web-based API for his online shop. The API